LAPORAN PRAKTIKUM

Modul 8

Disusun Untuk Memenuhi Tugas Kuliah

Mata Kuliah: Teknologi IoT

Dosen Pembimbing:

Agus Ambarwari, S.Pd., M.Kom.

Dr. Ir. Septaliansyah Dwi Putra

Oleh:

Alby Halim (25782031)

PROGRAM STUDI D4 TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGRI LAMPUNG

8.5.1 Praktikum 1: Peningkatan Ekosistem IoT Server dengan InfluxDB v2 dan Grafana pada Docker Compose

```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26200.9711]
(c) Microsoft Corporation. All rights reserved.

C:\Users\albih\iot-server>powershell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\albih\iot-server> New-Item -ItemType Directory -Force -Path influxdb/data, influxdb/config, grafana


    Directory: C:\Users\albih\iot-server\influxdb


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         11/2/2026   9:38 AM                data
d-----         11/2/2026   9:38 AM                config


    Directory: C:\Users\albih\iot-server


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         11/2/2026   9:38 AM                grafana


PS C:\Users\albih\iot-server>
```

Pada praktikum kali ini user akan meng-install atau pull influxdb sebelum itu, masuk ke folder kerja iot-server dan masukan perintah ini untuk buat folder influxdb

```
PS C:\Users\albih\iot-server> docker compose down
[+] down 3/3
 ✔Container node-red          Removed                                                    0.4s
 ✔Container mqtt-broker       Removed                                                    0.5s
 ✔Network iot-server_default  Removed                                                    0.2s
PS C:\Users\albih\iot-server>
```

```
      - mqtt

  # ============================================================
  # 3. Layanan Basis Data Deret Waktu: InfluxDB v2
  # ============================================================
  influxdb:
    image: influxdb:2.7
    container_name: influxdb2
    restart: unless-stopped
    ports:
      - "8086:8086"  # Port HTTP API & Web UI InfluxDB v2
    environment:
      # Inisialisasi otomatis akun administrator dan bucket awal
      DOCKER_INFLUXDB_INIT_MODE: setup
      DOCKER_INFLUXDB_INIT_USERNAME: admin
      DOCKER_INFLUXDB_INIT_PASSWORD: adminpassword
      DOCKER_INFLUXDB_INIT_ORG: my-org
      DOCKER_INFLUXDB_INIT_BUCKET: my-bucket
      DOCKER_INFLUXDB_INIT_RETENTION: 30d
    volumes:
      - ./influxdb/data:/var/lib/influxdb2
      - ./influxdb/config:/etc/influxdb2

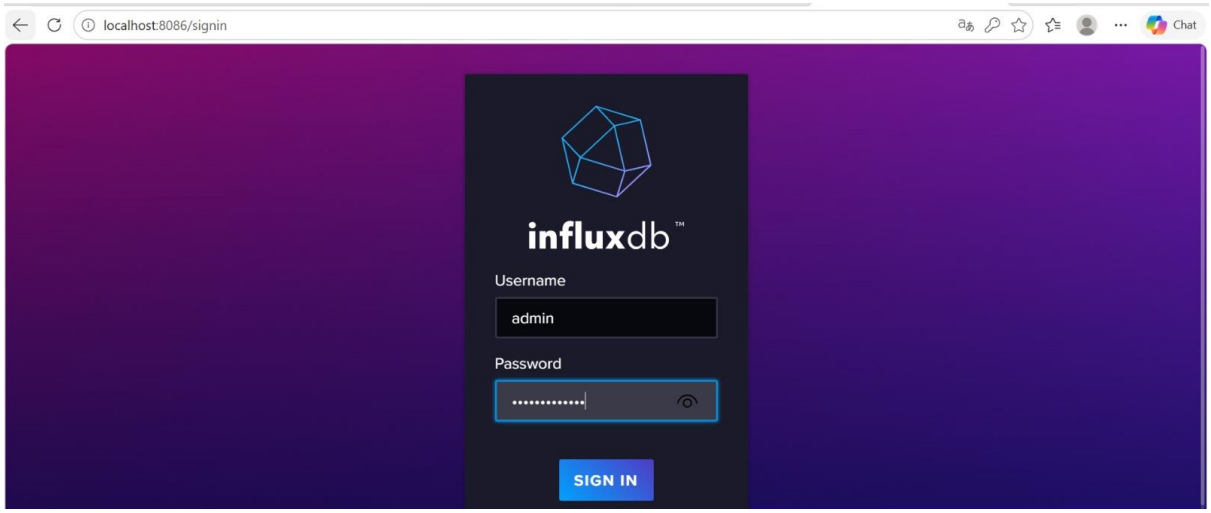
  # ============================================================
  # 4. Layanan Dasbor Visualisasi Industri: Grafana
  # ============================================================
  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    restart: unless-stopped
    ports:
      - "3000:3000"  # Port Antarmuka Dasbor Web Grafana
    environment:
      GF_SECURITY_ADMIN_USER: admin
```

kemudian di file Docker.Compose.yml lakukan edit dengan kode ini untuk dapat pull image influxdb dan layanan dasbor InfluxDB menggunakan grafana

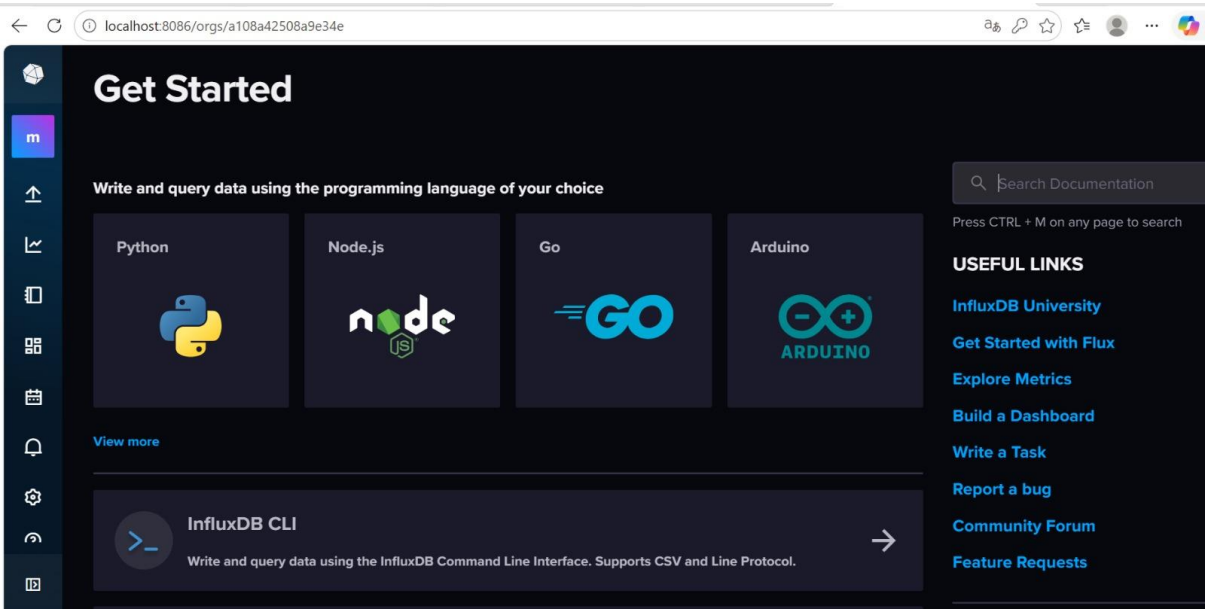


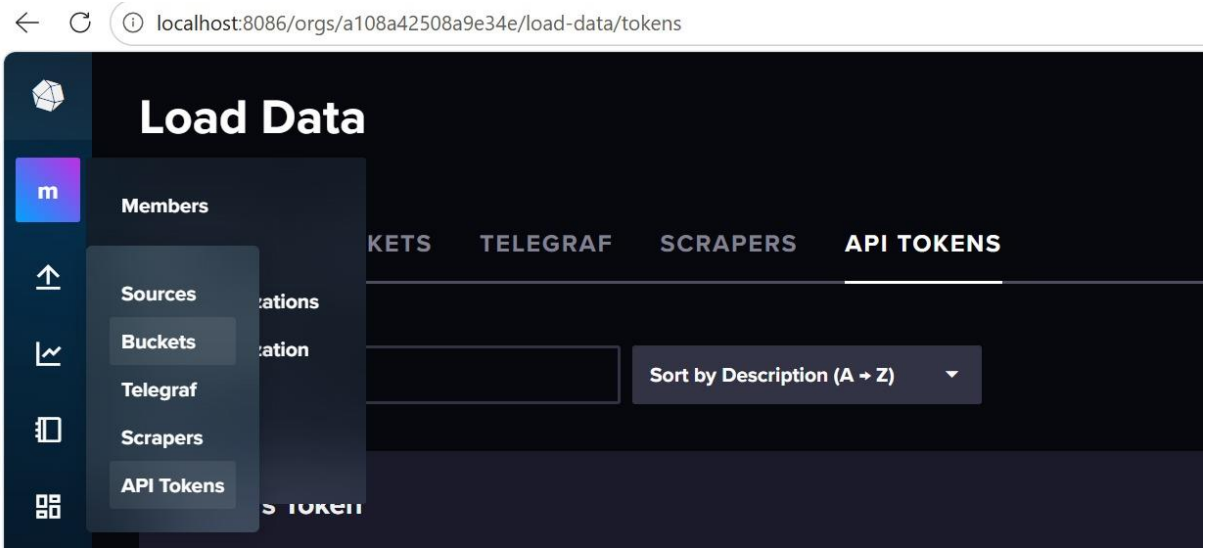
Setelah Compose down jalankan kembali Docker Compose Untuk update Docker yang sebelumnya telah di konfigurasi

8.5.2 Praktikum 2: Inisialisasi dan Manajemen Kredensial & API Token InfluxDB v2

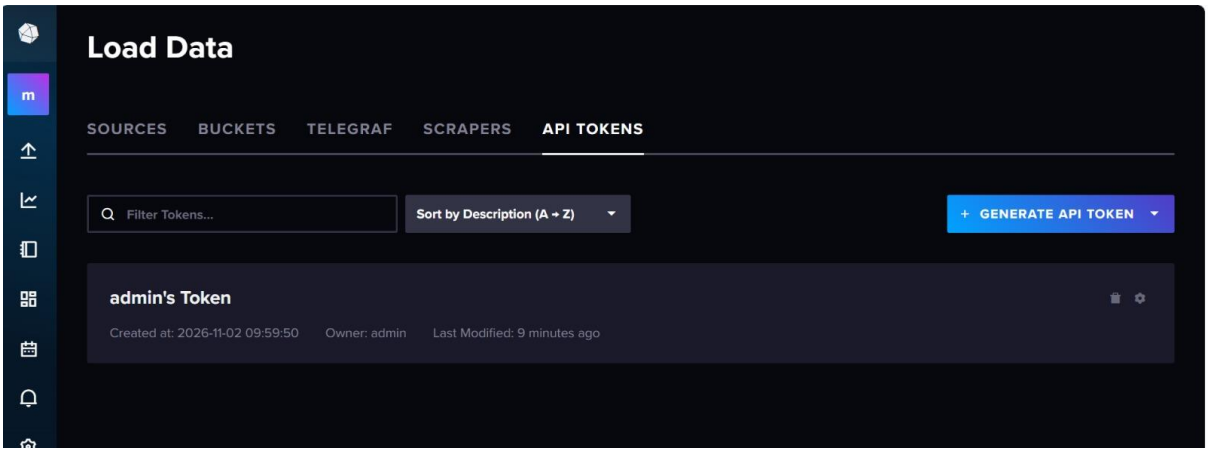


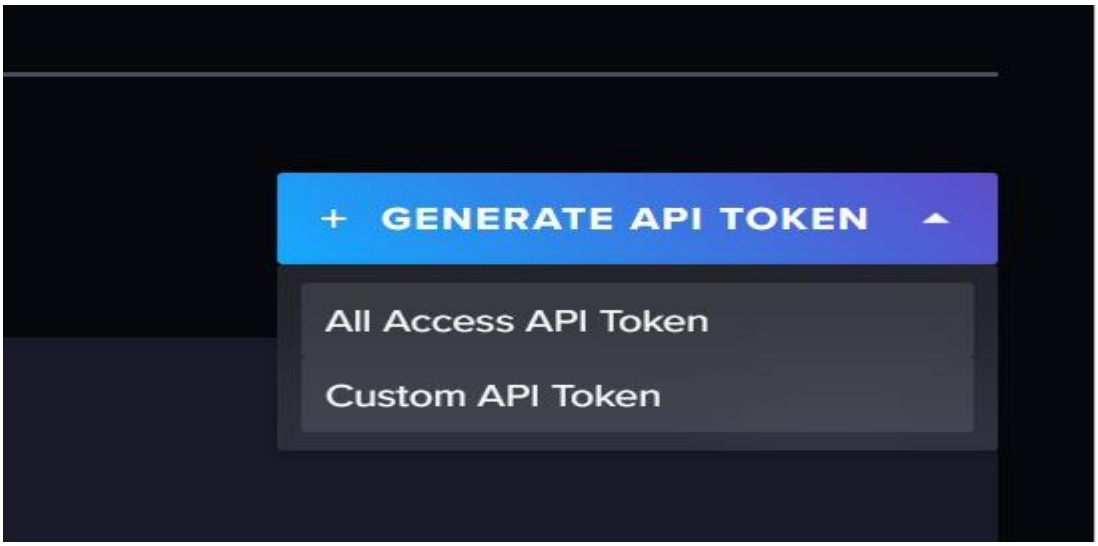
Sebelumnya user telah mengkonfigurasi file yml, lalu jika sudah, masuk ke browser lalu masukkan keyword ini untuk bisa masuk ke InfluxDB, untuk username dan pw sesuai isian masing masing user



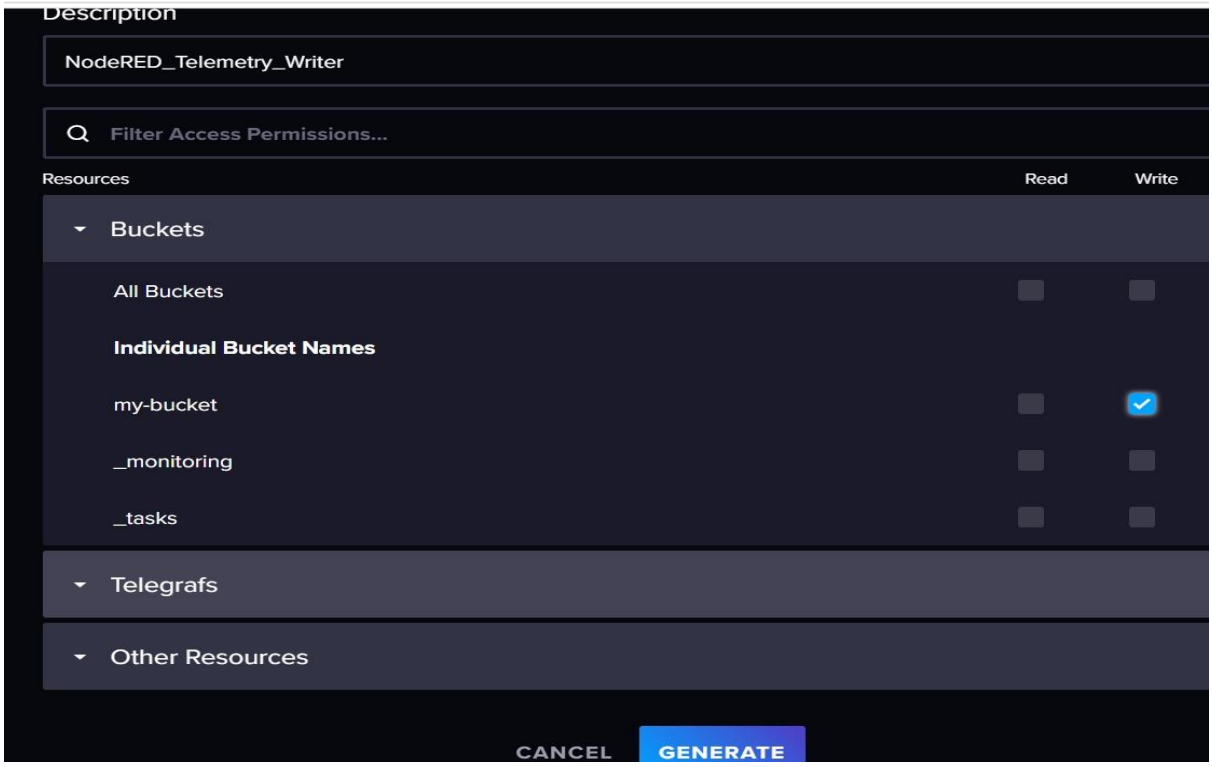


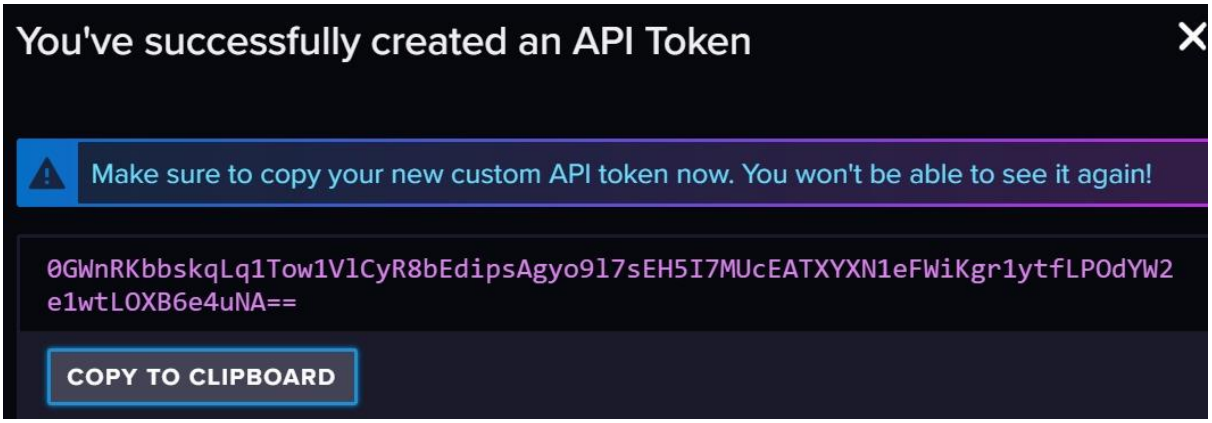
Setelah berhasil masuk, akan ada tampilan seperti ini, masuk ke icon silinder, lalu pilih buckets untuk melakukan konfg di praktikum ini





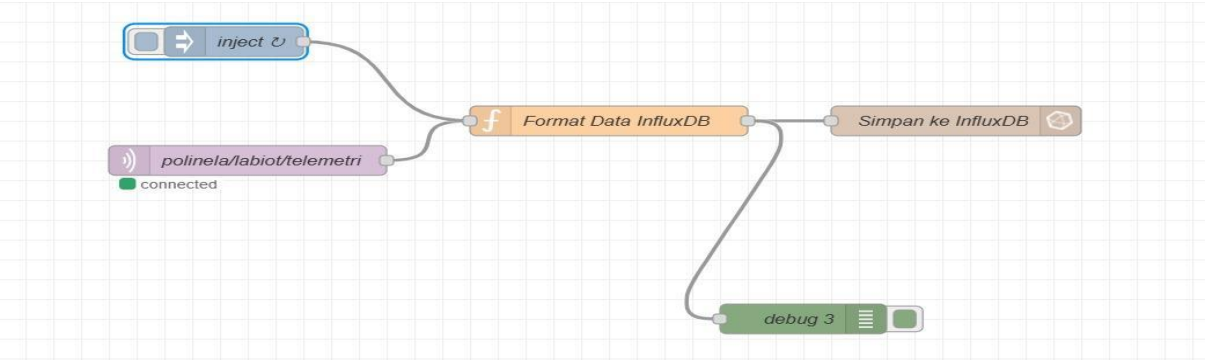
di sini user membuat api dengan cara klik generate lalu masuk ke pilihan custom api Token



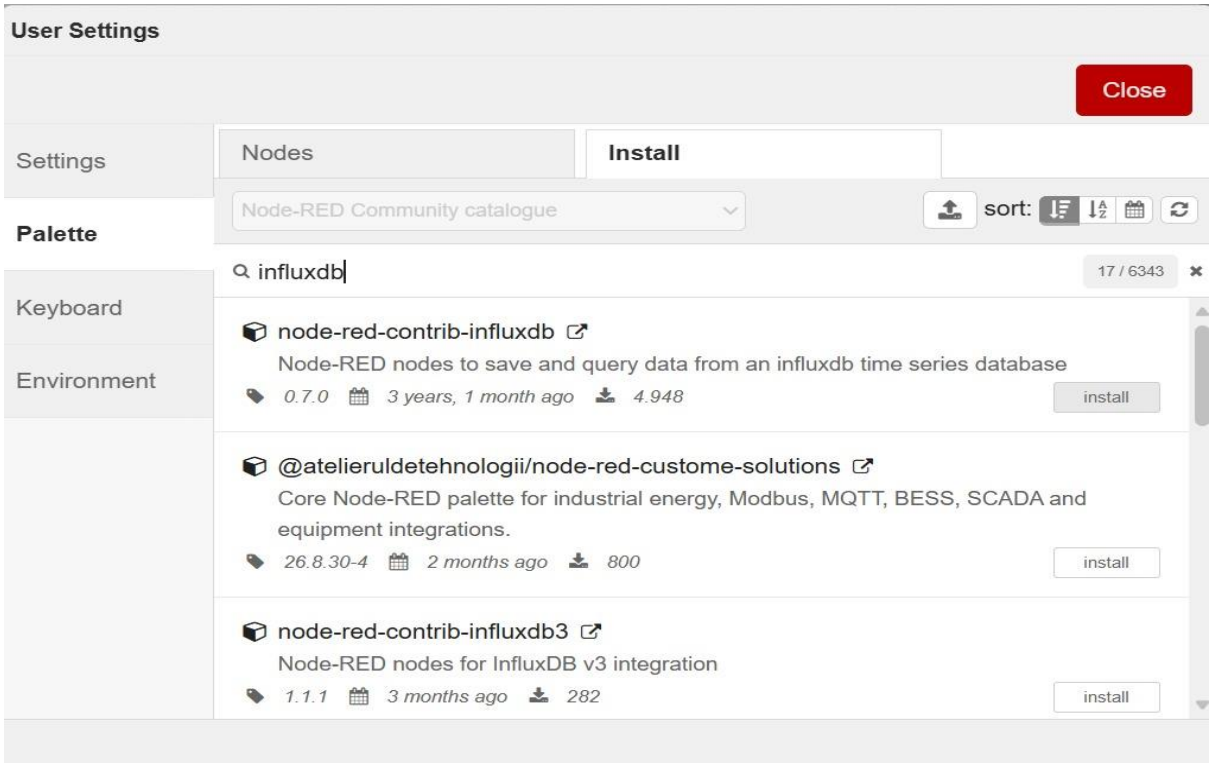


setelah itu ikuti settingan di atas setelah itu klik generate, nanti akan muncul api token khusus, dan jangan lupa untuk api token di salin dan di simpan di notepad agar tidak hilang, karena token akan digunakan di praktikum berikutnya

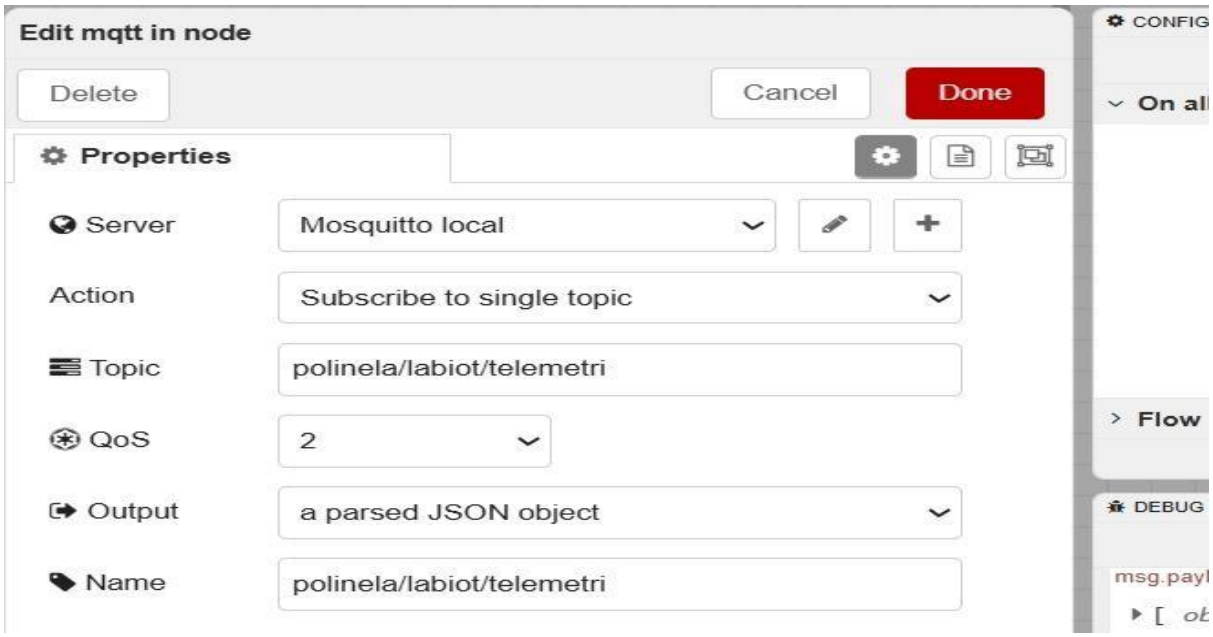
8.5.3 Praktikum 3: Instalasi Node InfluxDB dan Perancangan Pipeline Data pada Node-RED



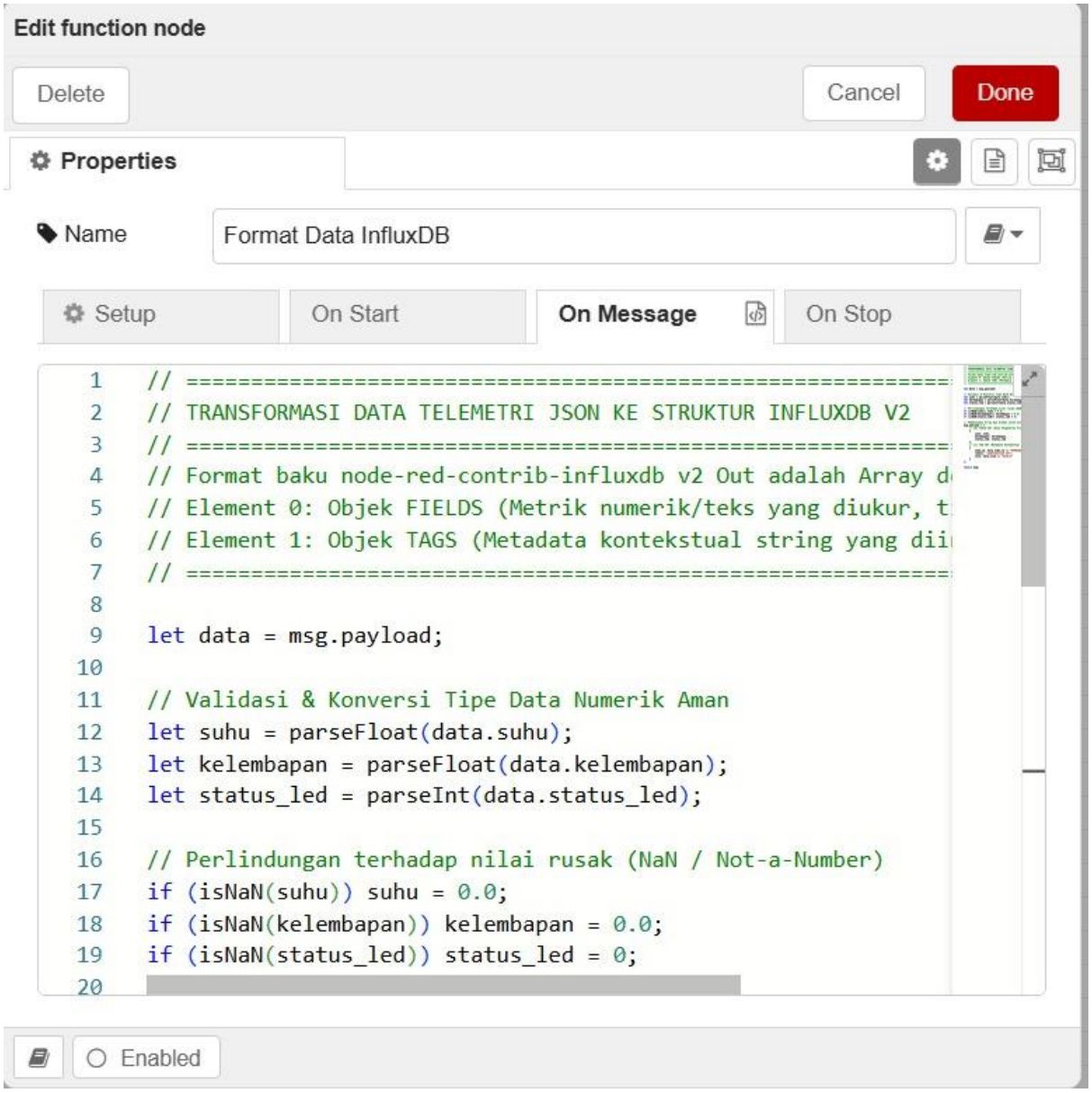
Rangkaian pallete untuk praktikum modul 8



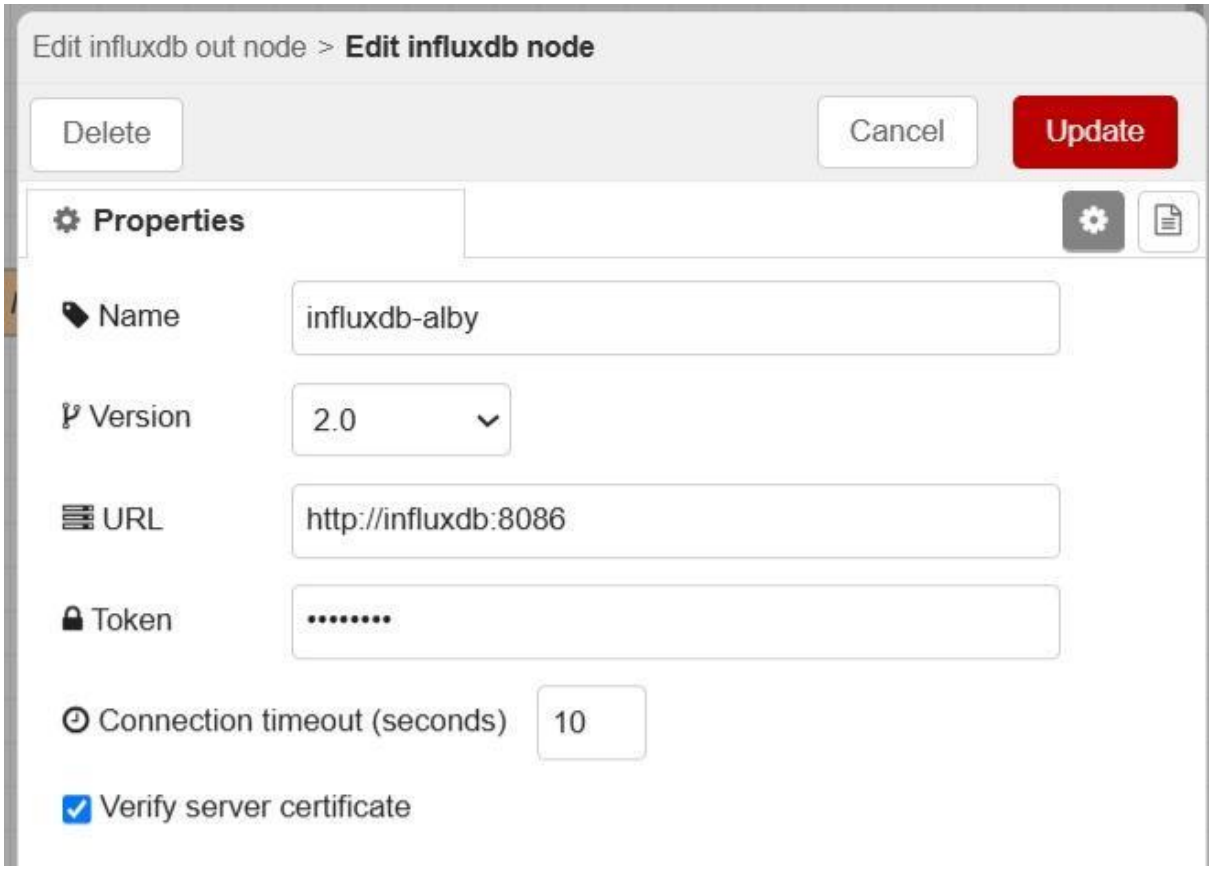
Di praktikum kali ini user membutuhkan pallete khusus yang harus di install terlebih dahulu, user dapat masuk ke settingan setelah itu manage pallete, setelah masuk ke tampilan ini user bisa meng-install InfluxDB untuk membuka beberapa pilihan pallete untuk InfluxDB



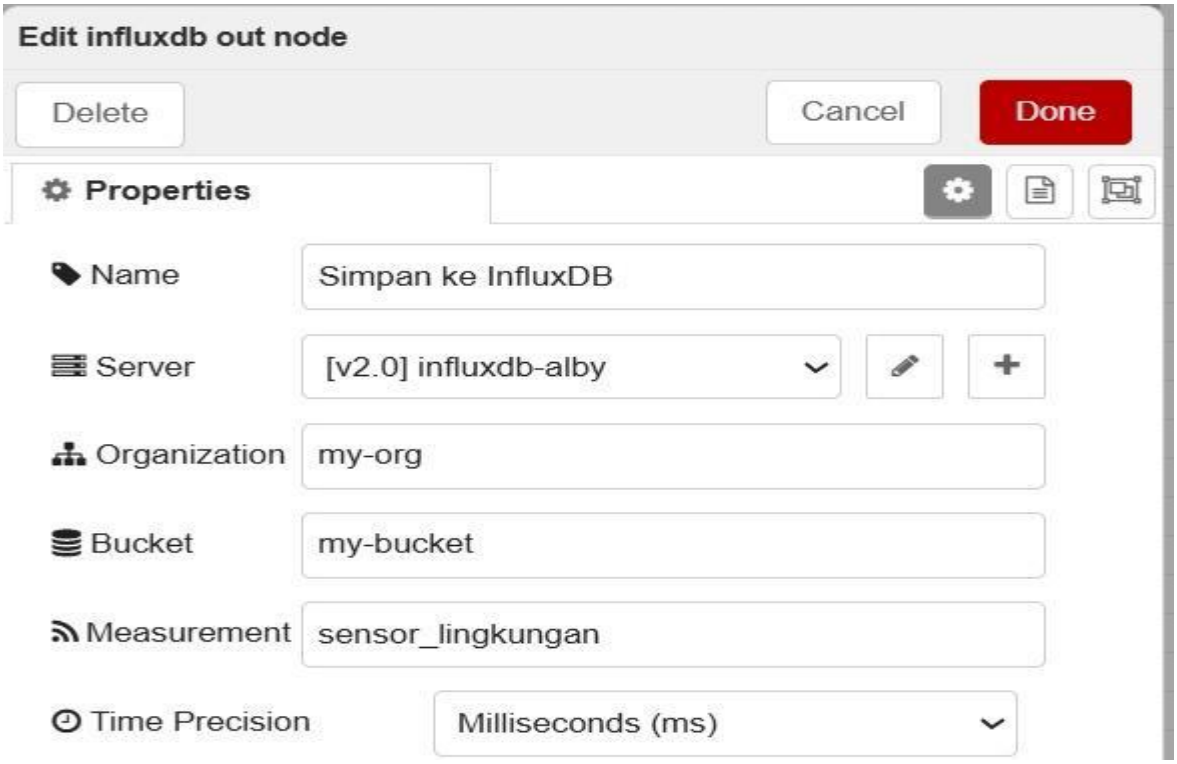
Ikuti topology di atas, lalu untuk mqtt ini di sini user dapat memggunakan server yang sebelumnya sudah connect di modul 7, dan juga topicnya sama seperti di modul 7



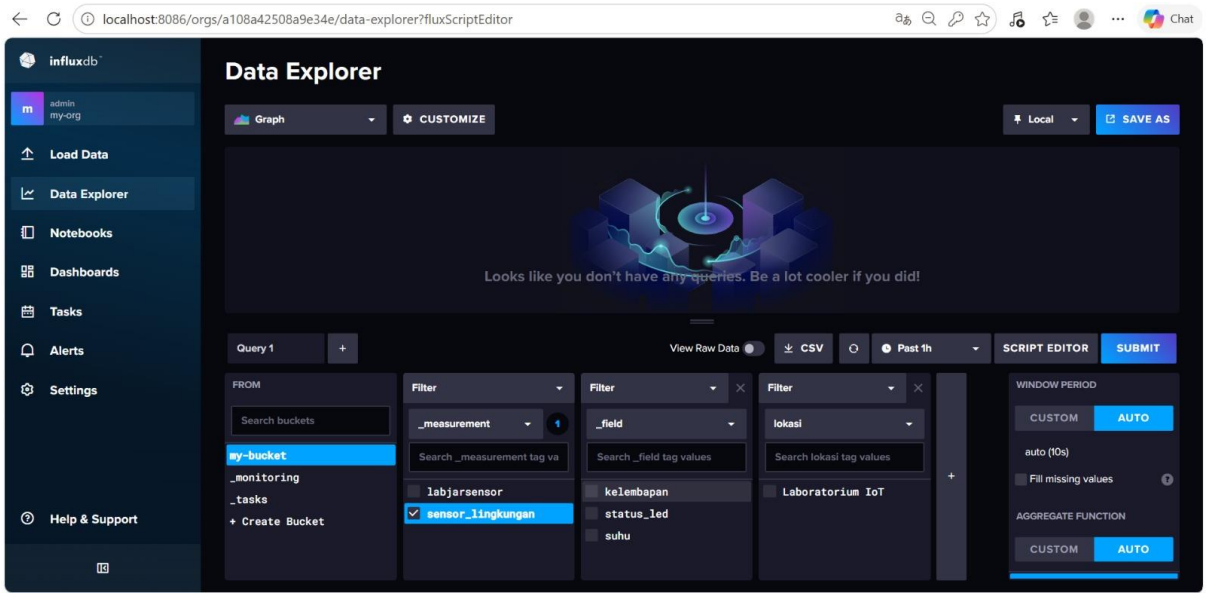
Pada pallete function user dapat memasukkan kode ini agar output keluaran akan sesuai nantinya



Pada influxdb out sebelum itu edit di node, lalu untuk version pastikan version di 2.0 dan untuk url di sesuaikan, ganti localhost menjadi InfluxDB, di bagian token salin token yang sebelum nya sudah di dapat dan di simpan di notepad

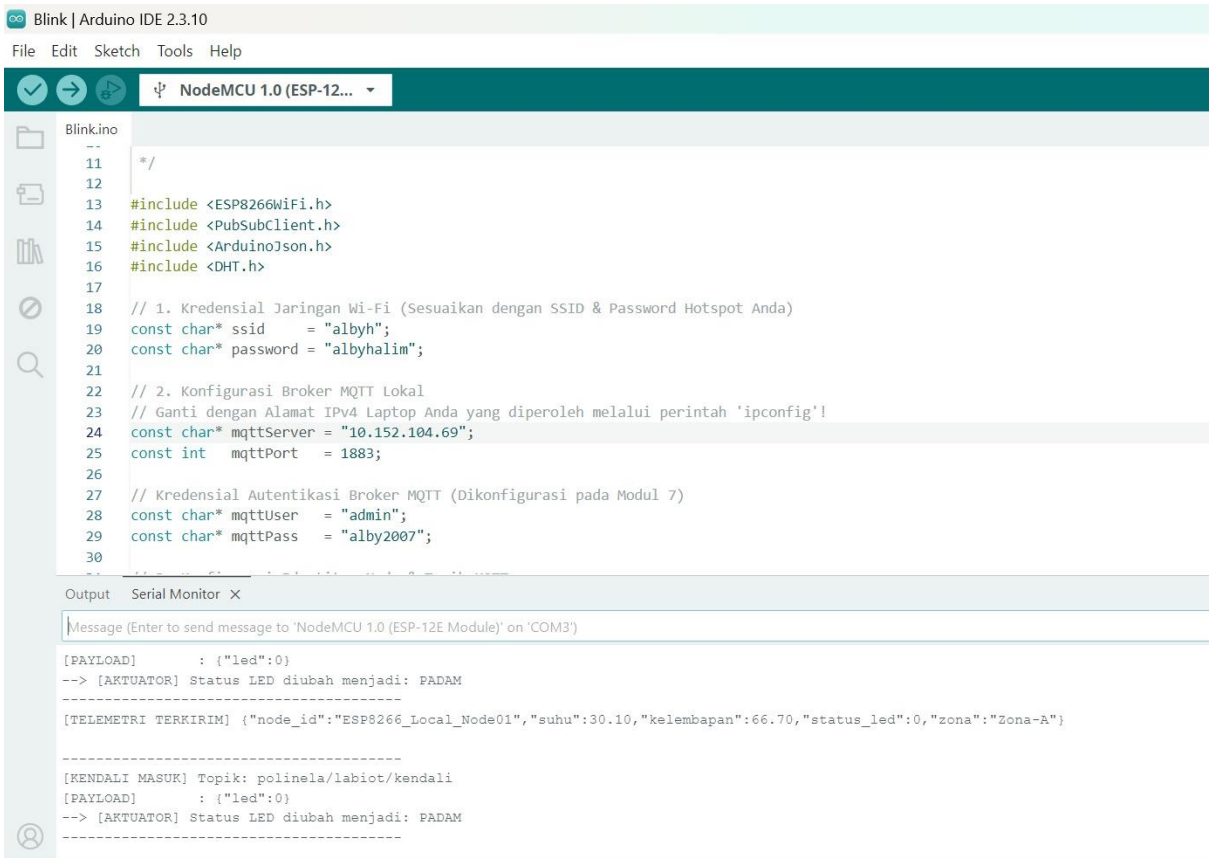


Pada bagian ini sesuaikan organization, dan bucket sesuai di localhost influxdb lalu jika semua sudah klik done

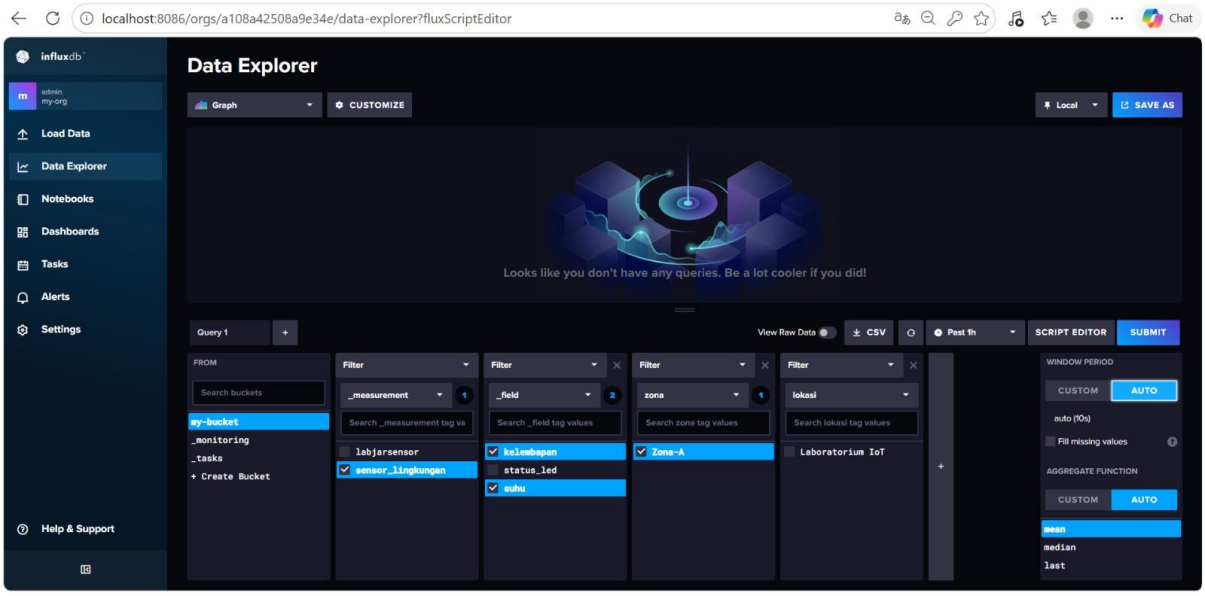


Masuk ke data explorer, lalu pilih bucket user yang di tambahkan tadi, nanti akan ada tampilan data monitoring disini jika berhasil semua pallete connect

8.5.4 Praktikum 4: Integrasi End-to-End NodeMCU ESP8266 dan Verifikasi Telemetri Time-Series

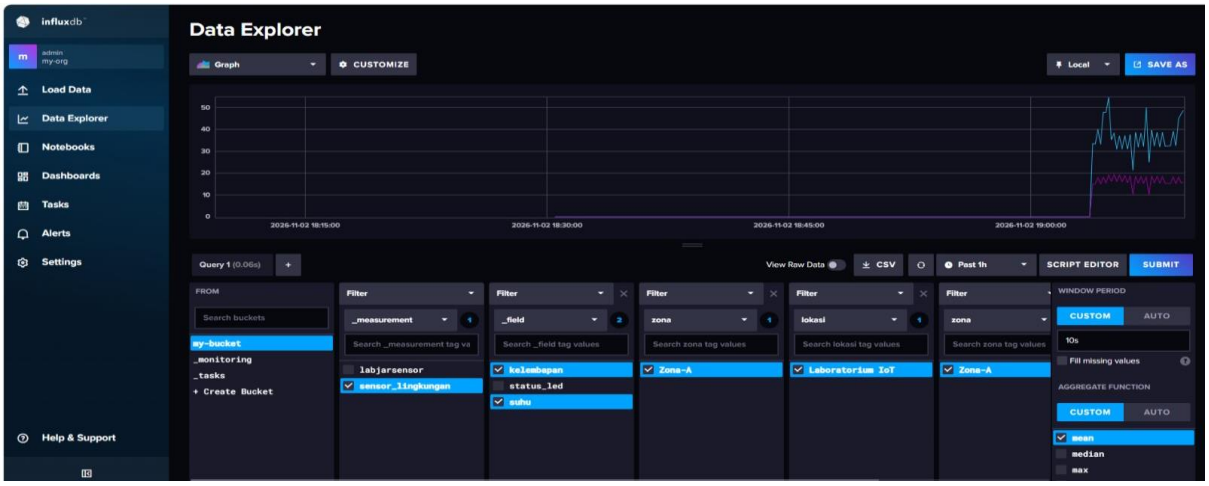


Setelah data pada influxdb sudah connect, selanjutnya buka arduino dan pastikan perangkat sudah jadi, lalu masukkan kode ini, nanti hasil dari monitoring ini akan dikirim ke data explorer di Influxdb

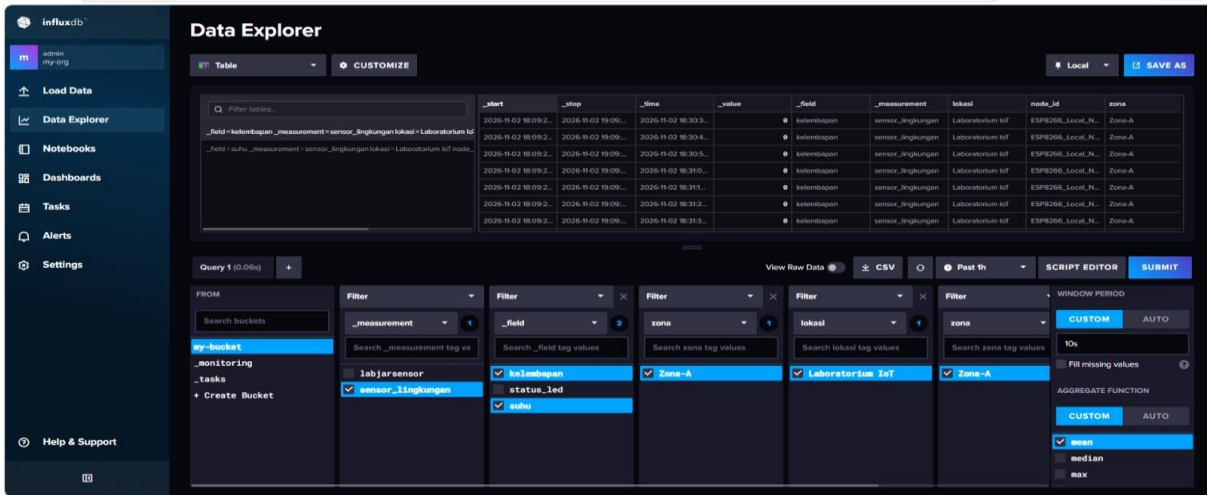


Ikuti settingan ini, lalu jika semua nya sudah klik summit

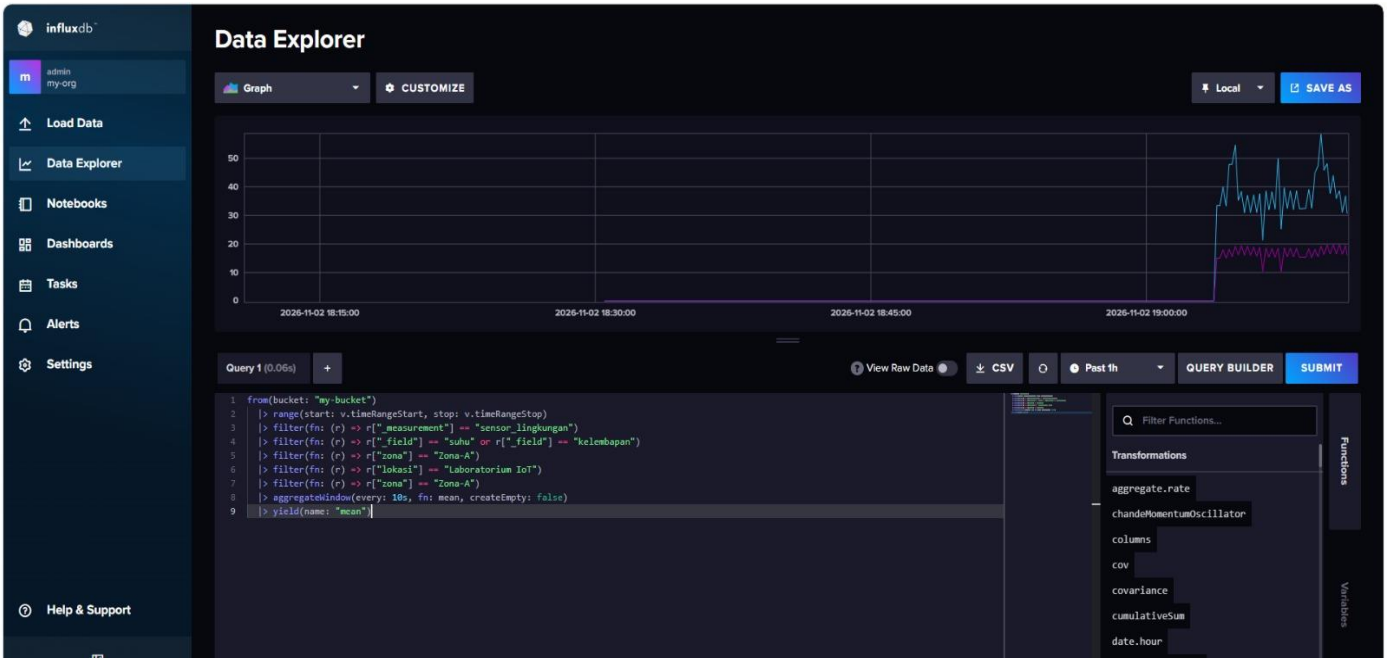
Graph



Tablet data



Kode flux


influxdb
admin
my-org
Load Data
Data Explorer
Notebooks
Dashboards
Tasks
Alerts
Settings
Help & Support
Data Explorer
Graph
CUSTOMIZE
Local
SAVE AS
50
40
30
20
10
0
2026-11-02 18:15:00
2026-11-02 18:30:00
2026-11-02 18:45:00
2026-11-02 19:00:00
Query 1 (0.06s)
View Raw Data
CSV
Past 1h
QUERY BUILDER
SUBMIT
from(bucket: "my-bucket")
|> range(start: v.timeRangeStart, stop: v.timeRangeStop)
|> filter(fn: (r) => r["_measurement"] == "sensor_lingkungan")
|> filter(fn: (r) => r["_field"] == "suhu" or r["_field"] == "kelembapan")
|> filter(fn: (r) => r["zona"] == "Zona-A")
|> filter(fn: (r) => r["lokasi"] == "Laboratorium IoT")
|> filter(fn: (r) => r["zona"] == "Zona-A")
|> aggregateWindow(every: 10s, fn: mean, createEmpty: false)
|> yield(name: "mean")
Filter Functions...
Transformations
aggregate.rate
chandeMomentumOscillator
columns
cov
covariance
cumulativeSum
date.hour
Functions
Variables

8.6 Latihan

Latihan 1: Analisis Kinerja & Efisiensi Penyimpanan (TSDB vs RDBMS)

Soal: Sebuah pabrik cerdas memasang 100 simpul sensor NodeMCU ESP8266. Setiap simpul mengirimkan 3 metrik float (suhu, kelembapan, getaran mesin) setiap 2 detik secara kontinu selama 30 hari penuh.

1. Jumlah total titik data selama 30 hari

a. Jumlah detik dalam 30 hari = 30 × 24 × 3.600 = 2.592.000 detik.

b. Jumlah sampel per simpul = 2.592.000 ÷ 2 = 1.296.000 sampel (pengiriman).

c. Jumlah sampel seluruh simpul = 100 × 1.296.000 = 129.600.000 sampel (pesan/baris).

d. Jumlah titik data (nilai metrik) = 129.600.000 × 3 metrik = **388.800.000 titik data**.

Jadi, sistem basis data harus menyimpan sekitar **388,8 juta titik data** (atau 129,6 juta baris/pesan telemetri, di mana setiap baris memuat 3 nilai metrik) dalam 30 hari. Laju penulisannya = 100 simpul ÷ 2 detik = 50 pesan/detik, atau 150 titik data/detik.

2. Rasio penghematan ruang penyimpanan

Perhitungan memakai 388.800.000 titik data.

a. Rasio ukuran = 150 ÷ 2,5 = **60 kali** lebih kecil (InfluxDB jauh lebih hemat).

b. Penghematan ruang = (58,32 − 0,972) ÷ 58,32 × 100% = **≈ 98,33%**.

c. Selisih ruang yang dihemat ≈ 58,32 GB − 0,972 GB = **≈ 57,35 GB** hanya untuk 30 hari data.

Artinya, InfluxDB hanya membutuhkan sekitar 1/60 (≈ 1,67%) dari ruang disk yang dipakai RDBMS. Angka ini bahkan lebih baik dari klaim umum 80–90% pada modul karena angka 2,5 byte pada soal merupakan kompresi yang sangat efisien.

3. Penyebab penurunan performa B-Tree (write degradation)

Indeks B-Tree dirancang untuk akses acak yang seimbang antara baca dan tulis, bukan untuk aliran tulis konstan bervolume besar. Penyebab utamanya:

a. **Penyeimbangan ulang pohon (page split & rebalancing).** Setiap baris baru harus disisipkan ke posisi yang tepat pada pohon terurut. Ketika halaman (page) penuh, terjadi *page split* dan penyeimbangan ulang yang memakan CPU dan I/O, sehingga makin sering terjadi seiring pertumbuhan tabel.

b. **Random I/O dan fragmentasi.** Penyisipan menyebar ke banyak halaman di disk sehingga menimbulkan *random write*, halaman setengah kosong, dan fragmentasi indeks. Hal ini jauh lebih lambat dibanding penulisan sekuensial.

c. **Write amplification.** Satu baris data memicu pembaruan pada tabel utama dan seluruh indeks sekunder, ditambah pencatatan WAL/redo log. Satu penulisan logis berubah menjadi banyak penulisan fisik.

d. **Overhead transaksi ACID.** Setiap penyisipan membawa biaya penguncian baris (*row locking*), manajemen *transaction log*, dan *MVCC* yang tidak diperlukan oleh data sensor yang bersifat *append-only* dan tidak pernah diubah.

e. **Indeks tidak lagi muat di RAM.** Setelah tabel mencapai ratusan juta baris, bagian indeks yang aktif tidak lagi muat di *buffer pool* sehingga banyak operasi harus membaca disk (*cache miss*) dan throughput turun tajam.

Sebaliknya, InfluxDB memakai arsitektur TSM/LSM yang menulis data secara sekuensial ke WAL dan *cache*, lalu menggabungkannya (*compaction*) ke berkas TSM yang *immutable* dalam blok kolom terurut waktu. Pola ini sangat cocok dengan data sensor sehingga laju tulisnya tetap stabil.

Latihan 2: Fenomena Cardinality Explosion pada InfluxDB Tags

Pengembang pemula memasukkan nilai acak presisi tinggi seperti UUID (uuid=a8f9-c412...) atau pembacaan sensor pecahan (suhu_str=29.84512) ke dalam Tag Set, bukan Field Set.

1. Mekanisme Inverted Index pada Tags dan penyebab Cardinality Explosion

Seluruh pasangan kunci–nilai pada Tag Set otomatis dimasukkan ke dalam *Inverted Index*, yaitu struktur yang memetakan setiap nilai tag ke daftar *series* (deret waktu) yang memilikinya, mirip indeks di belakang buku. Contohnya, nilai tag zona=Zona-A menunjuk ke daftar semua *series* yang berasal dari Zona-A. Karena itu, pencarian berbasis tag sangat cepat.

Sebuah *series* didefinisikan oleh kombinasi unik dari measurement + seluruh pasangan tag. Jumlah series total (*series cardinality*) kira-kira sama dengan hasil kali jumlah nilai unik pada setiap tag:

$$\text{Cardinality} \approx |\text{node_id}| \times |\text{lokasi}| \times |\text{zona}| \times \dots$$

Bila nilai tag selalu unik di setiap sampel (UUID atau suhu pecahan), setiap pesan membentuk series baru dan entri baru di Inverted Index. Setelah 129,6 juta pesan (hitungan Latihan 1) jumlah series ikut mencapai ±129,6 juta, padahal seharusnya hanya 100 series (satu per node). Pertumbuhan tanpa batas inilah yang disebut **Cardinality Explosion**. Selain itu, tiap series hanya berisi satu titik data, sehingga kompresi dan keunggulan struktur deret waktu hilang sama sekali.

2. Dampak terhadap RAM dan stabilitas kontainer Docker InfluxDB

a. **Konsumsi RAM membengkak.** Indeks series disimpan dan di-*cache* di memori kerja sehingga pemakaian RAM naik seiring jumlah series unik, bukan jumlah data.

b. **Kinerja kueri dan penulisan menurun.** Pencarian di indeks raksasa menjadi lambat, *compaction* dan *startup* (pemuatan indeks) makin lama, serta latensi penulisan naik.

c. **OOM Kill pada kontainer.** Jika kontainer dibatasi memori (atau RAM host habis), kernel Linux akan menghentikan proses InfluxDB (*Out of Memory Kill*, exit code 137).

Kontainer *crash* atau masuk *restart loop*, dan karena indeks harus dimuat ulang saat start, proses yang sama bisa berulang.

d. **Layanan lain ikut terdampak.** Pada satu host Docker yang sama, kekurangan memori dapat memengaruhi Mosquitto, Node-RED, dan Grafana, sehingga seluruh pipeline IoT berhenti dan data baru hilang.

3. Pedoman baku menentukan Tag atau Field

(1) Tanyakan "apakah saya akan memfilter atau mengelompokkan data berdasarkan variabel ini, dan apakah jumlah nilainya terbatas?". Jika ya, jadikan Tag. (2) Jika nilainya angka hasil pengukuran, acak, atau selalu unik, wajib dijadikan Field. (3) Bila ragu, jadikan Field karena Field aman dari risiko kardinalitas. (4) Perkirakan kardinalitas total sebelum merancang skema, dan pantau dengan fungsi Flux influxdb.cardinality().

Latihan 3: Arsitektur Jaringan Docker dan Resolusi Nama Layanan DNS

Pada node influxdb out Node-RED, alamat server diisi http://influxdb:8086, sedangkan pada browser laptop dibuka http://localhost:8086.

1. Mengapa localhost:8086 di Node-RED menyebabkan koneksi gagal

Setiap kontainer Docker memiliki *network namespace* sendiri yang terisolasi. Di dalam kontainer, alamat localhost (127.0.0.1) menunjuk ke **kontainer itu sendiri**, bukan ke laptop (host) dan bukan ke kontainer lain. Saat node influxdb out berjalan di dalam kontainer Node-RED dan mengakses http://localhost:8086, ia mencari layanan di port 8086 milik kontainer Node-RED sendiri. Karena tidak ada proses yang mendengarkan di sana, koneksi ditolak (*Connection Refused / ECONNREFUSED*) atau habis waktu (*ETIMEDOUT*).

Sebaliknya, browser laptop berjalan di host, sehingga localhost:8086 benar. Port 8086 pada host telah dipetakan ke kontainer InfluxDB melalui baris ports: "8086:8086" pada docker-compose.yml. Maka dari dalam Node-RED, alamat yang benar adalah nama layanan **influxdb** (nama service di Compose), yaitu http://influxdb:8086.

2. Peran Docker Embedded DNS Resolver (127.0.0.11) dan jaringan bridge iot-net

a. **Jaringan bridge iot-net.** Docker Compose membuat jaringan virtual (*user-defined bridge*) yang menghubungkan seluruh kontainer (Mosquitto, Node-RED, InfluxDB, Grafana) yang tergabung di dalamnya. Kontainer di jaringan yang sama mendapat alamat IP privat (misalnya 172.x.x.x) dan dapat saling berkomunikasi langsung, sedangkan kontainer di luar jaringan tidak bisa mengaksesnya. Jaringan ini terisolasi dari jaringan lain dan dari dunia luar.

b. **Docker Embedded DNS Resolver (127.0.0.11).** Pada user-defined bridge, Docker menjalankan server DNS internal yang dapat diakses oleh tiap kontainer pada alamat 127.0.0.11. Ketika Node-RED meminta nama influxdb, kueri DNS dikirim ke 127.0.0.11, yang menerjemahkan nama layanan atau nama kontainer menjadi alamat IP kontainer InfluxDB saat itu.

c. **Manfaat resolusi nama.** Alamat IP kontainer bersifat dinamis dan bisa berubah setiap kali kontainer dibuat ulang (*docker compose down/up*). Dengan nama layanan, konfigurasi tetap valid tanpa perlu mengubah IP secara manual.

d. **Tidak perlu mengekspos port ke publik.** Komunikasi antar-kontainer dalam iot-net memakai port internal kontainer (misalnya 8086 pada InfluxDB) secara langsung, sehingga tidak butuh pemetaan ports ke host. Pemetaan port hanya diperlukan untuk akses dari luar, misalnya browser laptop. Untuk produksi, ports pada InfluxDB bisa dihapus sehingga hanya kontainer di iot-net yang dapat mengaksesnya, dan ini memperkecil permukaan serangan (*attack surface*).

Latihan 4: Strategi Retensi Data & Agregasi Berjenjang (Downsampling)

Data telemetri frekuensi tinggi (per detik) penting untuk investigasi gangguan dalam 7 hari terakhir, namun untuk laporan KPI tahunan manajemen hanya butuh rata-rata per jam.

1. Mengapa menyimpan data presisi detik selama bertahun-tahun adalah pemborosan

a. **Pemborosan ruang disk.** Satu sensor per detik menghasilkan 86.400 titik per hari atau ±31,5 juta titik per tahun. Rata-rata per jam hanya 24 titik per hari (±8.760 per tahun), sehingga data mentah ±3.600 kali lebih banyak dari yang dibutuhkan untuk KPI.

b. **Pemborosan komputasi.** Kueri tahunan terhadap miliaran titik mentah lambat, memakan CPU, RAM, dan I/O, serta membuat dasbor Grafana lambat.

c. **Nilai informasi menurun seiring waktu.** Detail per detik berguna untuk *root cause analysis* jangka pendek, tetapi untuk analisis tren jangka panjang cukup berupa agregat (mean, min, max).

d. **Biaya dan pemeliharaan lebih tinggi.** Perlu disk yang lebih besar, *backup* lebih lama, dan *compaction* lebih berat.

2. Konsep Retention Policy pada Bucket InfluxDB v2

Retention Policy (periode retensi) adalah aturan lama penyimpanan data pada sebuah *bucket*. Pada InfluxDB v2 pengaturan ini menjadi properti bucket (kolom *Delete Data*, misalnya Older than 7 days, atau Never). Data yang umurnya melebihi batas retensi dihapus otomatis oleh sistem, tanpa kueri DELETE manual.

3. Strategi Dual-Bucket dengan InfluxDB Task (Flux)

Alurnya: Node-RED menulis hanya ke bucket_raw_7d. Sebuah **InfluxDB Task** (skrip Flux terjadwal) berjalan setiap jam, membaca data mentah satu jam terakhir, menghitung rata-ratanya, lalu menulis hasilnya ke bucket_aggregated_1y dengan fungsi to(). Setelah 7 hari data mentah terhapus otomatis oleh retensi, sedangkan agregatnya tetap tersimpan 1 tahun.

Contoh skrip Flux Task:

```
option task = {name: "downsample_telemetri_1h", every: 1h}

from(bucket: "bucket_raw_7d")
  |> range(start: -task.every)
  |> filter(fn: (r) => r._measurement == "monitoring_lingkungan")
  |> filter(fn: (r) => r._field == "suhu" or r._field == "kelembapan")
  |> aggregateWindow(every: 1h, fn: mean, createEmpty: false)
  |> to(bucket: "bucket_aggregated_1y", org: "my-org")
```

Dengan strategi ini, ruang penyimpanan hemat (±99,9% lebih sedikit titik untuk data jangka panjang), kueri tahunan cepat karena hanya membaca data agregat, dan data detail tetap tersedia selama 7 hari untuk investigasi. Pengelolaannya otomatis tanpa intervensi manual.